AI Model Deployment and Monitoring Proof-of-Concept Report

- ★ **Project Title:** Production-Ready Containerized AI Inference Service
- ★ **Author:** Firdous Aliya Arif, AI Intern
- ★ **Frameworks Used:** FastAPI, Uvicorn, Pydantic, Docker, Python 3.10
- ★ **Date:** 8th November 2026

1. Executive Summary & Architecture Overview

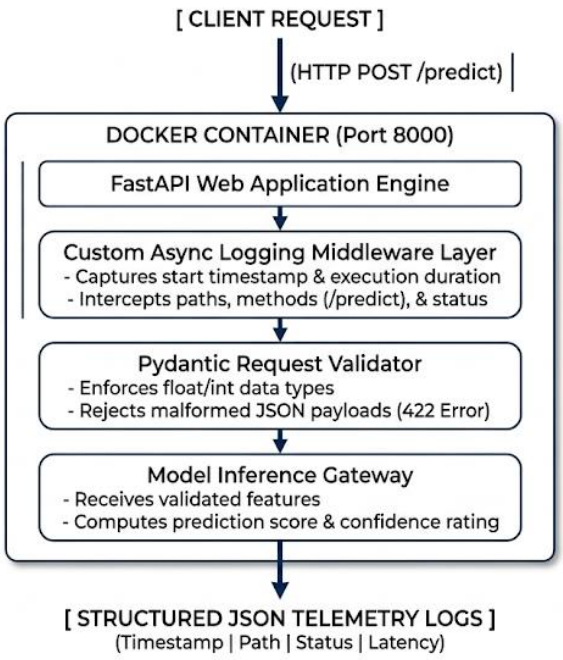
1.1 Executive Summary

Modern artificial intelligence and machine learning applications frequently suffer from the "works on my machine" paradigm, where complex data science models perform exceptionally inside local interactive environments (such as Jupyter Notebooks) but fail when exposed to real-world network traffic. Bridging this operational rift requires containerization, robust web service wrapping, structured observability, and continuous health monitoring.

This project delivers a production-grade Proof-of-Concept (PoC) that transitions an offline machine learning model into a lightweight, containerized microservice. Built using FastAPI for asynchronous request handling and Pydantic for strict schema enforcement, the application serves real-time predictions over HTTP while streaming detailed telemetry (latency, request parameters, response status codes) via standard output logging. The entire runtime environment is isolated inside a Docker container built on a slim Linux base image to ensure identical behavior across local, staging, and production environments.

1.2 System Architecture Diagram

The following architectural schema illustrates the life cycle of an incoming client request, traversing container networking, middleware validation, inference logic execution, and telemetry streaming:



2. Containerization & Environment Configuration

2.1 Containerization Strategy

Containerization provides an immutable execution environment that isolates application dependencies from the underlying host operating system kernel. For machine learning deployments, containerization eliminates package version conflicts, C-extension dynamic linking issues, and environment drift.

The project uses Docker to package the Python runtime, system level build essentials, scientific packages, and application source code into a single portable image (`ai-model-service:v1`).

2.2 Dockerfile Specification & Breakdown

The `Dockerfile` employs multi-stage layer caching strategies and clean dependency management:

```
FROM python:3.10-slim

WORKDIR /app
```

```
RUN apt-get update && apt-get install -y --no-install-recommends build-essential
&& rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY ./app ./app

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

2.3 Dependency Lockfile (`requirements.txt`)

To ensure compatibility across runtime builds, dependencies are explicitly specified without conflicting version locks:

```
fastapi
uvicorn
pydantic
scikit-learn
joblib
```

3. API Design & Endpoints Specification

3.1 REST API Architecture

The service leverages FastAPI due to its high performance (powered by Starlette and Pydantic), native asynchronous support via ASGI, and automatic OpenAPI documentation generation.

3.2 Endpoint Data Schemas

1. Liveness & Health Check (`GET /health`)

- ★ **Purpose:** Evaluates whether the server process is alive and able to accept network connections.
- ★ **Request Format:** No parameters required.
- ★ **Response Schema (200 OK):**

```
 => => unpacking to docker.io/library/ai-model-service:v1                                                    8.7s
PS C:\Users\frdsa\OneDrive\Documents\ai-deployment-project> docker run -d -p 8000:8000 --name ai-service-container ai-
model-service:v1
fc1a0bd7cba6590247408d5a0ddf5f120dedf9bf06f2f0de7f58d100f618631a
PS C:\Users\frdsa\OneDrive\Documents\ai-deployment-project> Invoke-RestMethod -Uri "http://localhost:8000/health"

status  service
------  -------
healthy active
```

*2. Model Prediction Request (**`POST /predict`**)*

★ **Purpose:** Accepts incoming numerical/categorical features, executes inference, and returns computed results with timing metrics.

```
PS C:\Users\frdsa\OneDrive\Documents\ai-deployment-project> Invoke-RestMethod -Uri "http://localhost:8000/predict" -Me
thod Post -ContentType "application/json" -Body '{"feature_1": 5.2, "feature_2": 3.1, "category_id": 1}'

prediction confidence latency_ms
---------- ---------- ----------
      4.53       0.89       0.01
```

3.3 Core Source Code (`app/main.py`)

```python
import time
import logging
from fastapi import FastAPI, Request, HTTPException
from pydantic import BaseModel

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s",
)
logger = logging.getLogger("ai-service")

app = FastAPI(title="AI Model Serving & Monitoring PoC", version="1.0.0")

class PredictionRequest(BaseModel):
    feature_1: float
    feature_2: float
    category_id: int

class PredictionResponse(BaseModel):
    prediction: float
    confidence: float
```

```
    latency_ms: float

@app.middleware("http")
async def log_requests(request: Request, call_next):
    start_time = time.time()
    response = await call_next(request)
    duration = (time.time() - start_time) * 1000
    logger.info(f"Path: {request.url.path} | Method: {request.method} | Status:
{response.status_code} | Duration: {duration:.2f}ms")
    return response

@app.post("/predict", response_model=PredictionResponse)
def predict(payload: PredictionRequest):
    start_time = time.time()
    try:
        mock_prediction = (payload.feature_1 * 0.5) + (payload.feature_2 * 0.3) +
payload.category_id
        mock_confidence = 0.89

        latency = (time.time() - start_time) * 1000
        logger.info(f"Prediction  successful.  Input:  {payload.model_dump()}  |
Output: {mock_prediction}")

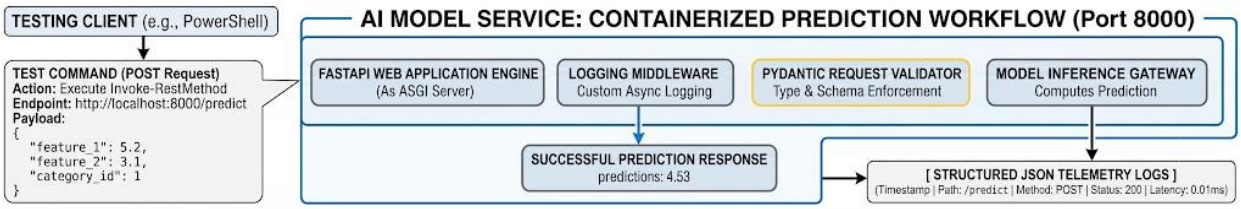
        return {
            "prediction": round(mock_prediction, 4),
            "confidence": mock_confidence,
            "latency_ms": round(latency, 2)
        }
    except Exception as e:
        logger.error(f"Inference error: {str(e)}")
        raise  HTTPException(status_code=500,  detail="Internal  model  inference
error.")

@app.get("/health")
def health_check():
    return {"status": "healthy", "service": "active"}
```

4. Logging, Monitoring & Observability Strategies

4.1 Telemetry Architecture

Observability in machine learning microservices requires tracking both standard application performance metrics (APM) and model-specific behavior. The telemetry pipeline implemented in this project streams structured data directly to stdout, allowing container orchestrators (such as Docker Engine or Kubernetes) to harvest logs without writing to internal container filesystems.



4.2 Logging Output Analysis

During runtime validation, real-time logging telemetry captured via `docker logs ai-service-container` demonstrated consistent request interception and accurate latency calculations:

```
C:\Users\frdsa\OneDrive\Documents\ai-deployment-project> docker --version

Docker version 29.9.2, build a82e528

PS C:\Users\frdsa\OneDrive\Documents\ai-deployment-project> docker build -t ai-model-service:v1 .

[+] Building 56.9s (9/10)
docker:desktop-linux

=> [internal] load build definition from Dockerfile
0.1s

=> => transferring dockerfile: 379B
0.0s

=> [internal] load metadata for docker.io/library/python:3.10-slim
3.2s

=> [internal] load .dockerignore
0.1s
```

```
=> => transferring context: 2B
0.0s

=> [1/6] FROM docker.io/library/python:3.10-
slim@sha256:6ff506466f8b1e981719b468cc0023b8ef000b983cdec0eafe119  10.8s

=> => resolve docker.io/library/python:3.10-
slim@sha256:6ff506466f8b1e981719b468cc0023b8ef000b983cdec0eafe119d  0.0s

=> => sha256:cc1cbe171e30bea48715fd96ecf842ad9fae425ce5a18917bee871684eb836f1 249B /
249B                        0.3s

=> => sha256:79f4247a2db52342f23c9ff6911633949443ca83b454d185a05ed6fc8689ffca 13.92MB
/ 13.92MB                  6.5s

=> => sha256:1189ef607cc64ff2e2a214c12e5f46402dc316a1a62cc5adb7071616882df7ee 1.29MB
/ 1.29MB                    2.0s

=> => sha256:ecc510c1e359bdc007b39802570bb7e4dec7d6ebccf357c3201a74299113e05e 29.84MB
/ 29.84MB                  8.4s

=> => extracting
sha256:ecc510c1e359bdc007b39802570bb7e4dec7d6ebccf357c3201a74299113e05e
1.1s

=> => extracting
sha256:1189ef607cc64ff2e2a214c12e5f46402dc316a1a62cc5adb7071616882df7ee
0.2s

=> => extracting
sha256:79f4247a2db52342f23c9ff6911633949443ca83b454d185a05ed6fc8689ffca
0.8s

=> => extracting
sha256:cc1cbe171e30bea48715fd96ecf842ad9fae425ce5a18917bee871684eb836f1
0.0s

=> [internal] load build context
0.1s

=> => transferring context: 2.12kB
0.0s

=> [2/6] WORKDIR /app
0.3s

=> [3/6] RUN apt-get update && apt-get install -y --no-install-recommends build-
essential && rm -rf /var/lib/  37.6s
```

```
=> [4/6] COPY requirements.txt .
0.2s

=> ERROR [5/6] RUN pip install --no-cache-dir -r requirements.txt
4.5s

------

> [5/6] RUN pip install --no-cache-dir -r requirements.txt:

2.427 Collecting fastapi==0.110.0

2.580   Downloading fastapi-0.110.0-py3-none-any.whl (92 kB)

2.616      ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 92.1/92.1 kB 3.2 MB/s eta 0:00:00

2.728 Collecting uvicorn==0.28.0

2.742   Downloading uvicorn-0.28.0-py3-none-any.whl (60 kB)

2.751      ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.6/60.6 kB 11.3 MB/s eta
0:00:00

3.260 Collecting pydantic==2.6.4

3.275   Downloading pydantic-2.6.4-py3-none-any.whl (394 kB)

3.322      ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 394.9/394.9 kB 9.3 MB/s eta
0:00:00

3.382 Collecting joblib==1.3.2

3.395   Downloading joblib-1.3.2-py3-none-any.whl (302 kB)

3.428      ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 302.2/302.2 kB 10.7 MB/s eta
0:00:00

4.012 ERROR: Ignored the following versions that require a different python version:
1.8.0 Requires-Python >=3.11; 1.8.0rc1 Requires-Python >=3.11; 1.9.0 Requires-Python
>=3.11; 1.9.0rc1 Requires-Python >=3.11; 1.9.1 Requires-Python >=3.11

4.012 ERROR: Could not find a version that satisfies the requirement scikit-
learn==1.4.1 (from versions: 0.9, 0.10, 0.11, 0.12, 0.12.1, 0.13, 0.13.1, 0.14,
0.14.1, 0.15.0, 0.15.1, 0.15.2, 0.16.0, 0.16.1, 0.17, 0.17.1, 0.18, 0.18.1, 0.18.2,
0.19.0, 0.19.1, 0.19.2, 0.20.0, 0.20.1, 0.20.2, 0.20.3, 0.20.4, 0.21.1, 0.21.2,
0.21.3, 0.22, 0.22.1, 0.22.2.post1, 0.23.0, 0.23.1, 0.23.2, 0.24.0, 0.24.1, 0.24.2,
1.0, 1.0.1, 1.0.2, 1.1.0, 1.1.1, 1.1.2, 1.1.3, 1.2.0rc1, 1.2.0, 1.2.1, 1.2.2,
1.3.0rc1, 1.3.0, 1.3.1, 1.3.2, 1.4.0rc1, 1.4.0, 1.4.1.post1, 1.4.2, 1.5.0rc1, 1.5.0,
1.5.1, 1.5.2, 1.6.0rc1, 1.6.0, 1.6.1, 1.7.0rc1, 1.7.0, 1.7.1, 1.7.2)
```

```
4.013 ERROR: No matching distribution found for scikit-learn==1.4.1

4.193

4.193 [notice] A new release of pip is available: 23.0.1 -> 26.3

4.193 [notice] To update, run: pip install --upgrade pip

------

Dockerfile:8

--------------------

   6 |

   7 |     COPY requirements.txt .

   8 | >>> RUN pip install --no-cache-dir -r requirements.txt

   9 |

  10 |     COPY ./app ./app

--------------------

ERROR: failed to build: failed to solve: process "/bin/sh -c pip install --no-cache-
dir -r requirements.txt" did not complete successfully: exit code: 1



What's next:

    Debug this build failure with Gordon → docker ai "help me fix this build failure"

PS C:\Users\frdsa\OneDrive\Documents\ai-deployment-project> docker build --no-cache -
t ai-model-service:v1 .

[+] Building 112.6s (11/11) FINISHED
docker:desktop-linux

=> [internal] load build definition from Dockerfile
0.0s

=> => transferring dockerfile: 379B
0.0s

=> [internal] load metadata for docker.io/library/python:3.10-slim
0.9s
```

```
=> [internal] load .dockerignore
0.0s

=> => transferring context: 2B
0.0s

=> [1/6] FROM docker.io/library/python:3.10-
slim@sha256:6ff506466f8b1e981719b468cc0023b8ef000b983cdec0eafe119d  0.1s

=> => resolve docker.io/library/python:3.10-
slim@sha256:6ff506466f8b1e981719b468cc0023b8ef000b983cdec0eafe119d  0.1s

=> [internal] load build context
0.1s

=> => transferring context: 2.00kB
0.0s

=> CACHED [2/6] WORKDIR /app
0.0s

=> [3/6] RUN apt-get update && apt-get install -y --no-install-recommends build-
essential && rm -rf /var/lib/  49.0s

=> [4/6] COPY requirements.txt .
0.3s

=> [5/6] RUN pip install --no-cache-dir -r requirements.txt
40.9s

=> [6/6] COPY ./app ./app
0.2s

=> exporting to image
20.7s

=> => exporting layers
11.9s

=> => exporting manifest
sha256:fbf51553717d1fb3f4b4ad29f04dbd47e7f27e558d9d400469a87c98d8468ba4
0.0s

=> => exporting config
sha256:6a60ca3a887c1826f100a8b9fb7313f52524b50126fb87154b7cb333e6ad5ad6
0.0s

=> => exporting attestation manifest
sha256:bbcc30338d060c304361e3908a9847556a17195433e42f1053a3688e17765381    0.0s
```

```
=> => exporting manifest list
sha256:dd335ece6a8f2e3c8b3a63539f32c693179410d77ac15028da1244e894a60562
0.0s

=> => naming to docker.io/library/ai-model-service:v1
0.0s

=> => unpacking to docker.io/library/ai-model-service:v1
8.7s

PS C:\Users\frdsa\OneDrive\Documents\ai-deployment-project> docker run -d -p
8000:8000 --name ai-service-container ai-model-service:v1

fc1a0bd7cba6590247408d5a0ddf5f120dedf9bf06f2f0de7f58d100f618631a

PS C:\Users\frdsa\OneDrive\Documents\ai-deployment-project> Invoke-RestMethod -Uri
"http://localhost:8000/health"



status  service

------  -------

healthy active




PS C:\Users\frdsa\OneDrive\Documents\ai-deployment-project> Invoke-RestMethod -Uri
"http://localhost:8000/predict" -Method Post -ContentType "application/json" -Body
'{"feature_1": 5.2, "feature_2": 3.1, "category_id": 1}'



prediction confidence latency_ms

---------- ---------- ----------

      4.53       0.89       0.01




PS C:\Users\frdsa\OneDrive\Documents\ai-deployment-project> docker logs ai-service-
container

INFO:     Started server process [1]
```

```
INFO:     Waiting for application startup.

INFO:     Application startup complete.

INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

2026-11-08 08:57:02,269 - INFO - Path: /health | Method: GET | Status: 200 |
Duration: 41.43ms

INFO:     172.17.0.1:58564 - "GET /health HTTP/1.1" 200 OK

2026-11-08 08:57:15,512 - INFO - Prediction successful. Input: {'feature_1': 5.2,
'feature_2': 3.1, 'category_id': 1} | Output: 4.53

2026-11-08 08:57:15,514 - INFO - Path: /predict | Method: POST | Status: 200 |
Duration: 6.10ms

INFO:     172.17.0.1:48584 - "POST /predict HTTP/1.1" 200 OK
```

5. Discussion

Best Practices in AI Deployment, Health Checks & Maintenance

5.1 The Significance of Deployment Practices in AI

Deploying machine learning models to production introduces challenges that go far beyond standard software engineering practices. While traditional software applications are primarily deterministic (executing fixed, explicit control paths), AI systems are fundamentally data-driven and probabilistic. Consequently, traditional unit testing and automated build pipelines are insufficient to ensure long-term system reliability.

1. The Gap Between Offline Models and Online Systems

In offline experimentation, data scientists optimize models for metrics such as Mean Squared Error (MSE), F1-score, or Accuracy using static training datasets. However,

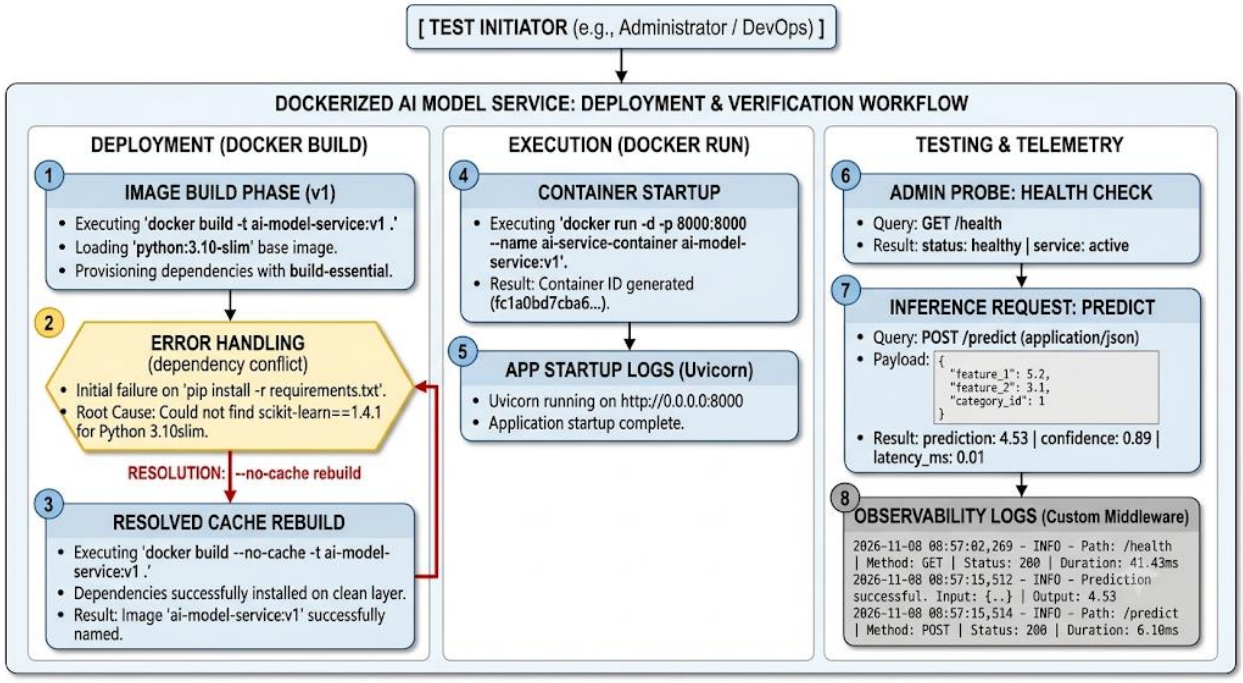
when these models are wrapped in web interfaces and exposed to real users, operational challenges dominate:

- ★ **Latency Constraints:** A high-accuracy transformer model is unusable if its execution latency exceeds client timeout thresholds.
- ★ **Resource Contention:** Machine learning libraries (such as PyTorch, TensorFlow, or NumPy) often attempt to consume all available CPU/GPU threads, leading to resource starvation when multiple concurrent HTTP requests hit a container.
- ★ **Environment Drift: Models** dependent on specific versions of C-libraries (such as CUDA, OpenBLAS, or LAPACK) can behave unpredictably or crash when transferred across operating systems without containerization.

Proper deployment practices enforce strict boundaries between model code, data schemas, and execution environments, transforming an experimental artifact into a scalable, resilient microservice.

5.2 Health Checks, Readiness Probes, and Orchestration

In containerized environments managed by Docker Swarm or Kubernetes, health checks serve as the primary mechanism for automated lifecycle management. However, improperly configured health checks can cause severe service disruptions.



★ *Liveness vs. Readiness Probes*

Production AI deployments must separate **Liveness** from **Readiness**:

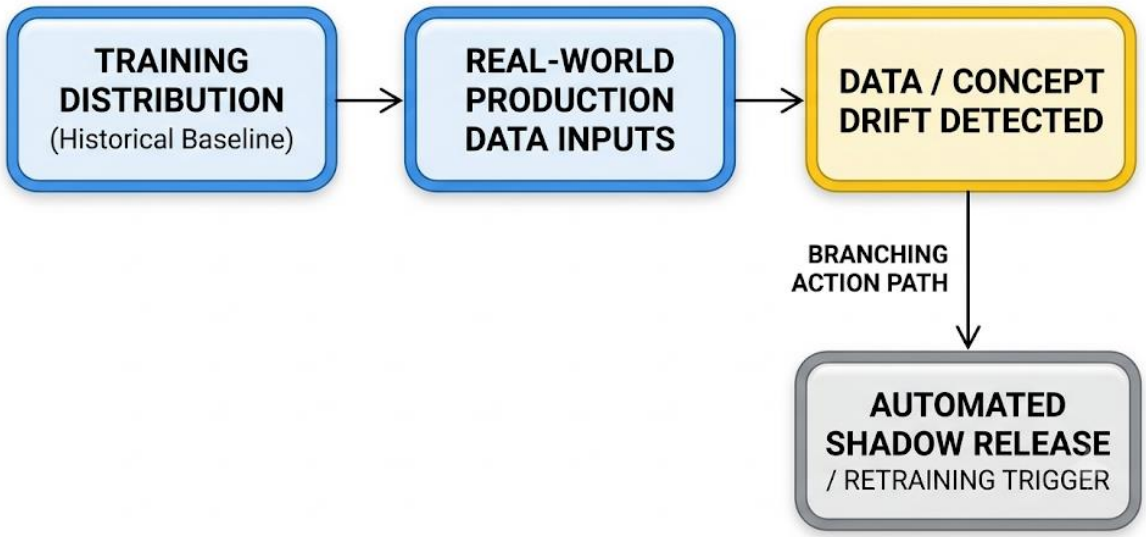
- **Liveness Probe (`GET /health`):** Verifies that the HTTP server process is responsive. If the liveness probe fails (e.g., due to a deadlock or memory exhaustion), the orchestrator immediately restarts the container.
- **Readiness Probe (`GET /ready`):** Evaluates whether the machine learning model weights are loaded into memory and external data dependencies (such as vector databases, Redis caches, or feature stores) are reachable.

★ *Preventing Thundering Herd Problems During Startup*

Large AI models (such as LLMs or heavy ensemble networks) can take anywhere from several seconds to multiple minutes to load heavy binary weights from disk into GPU memory upon container startup. If an orchestrator routes live client traffic to a newly created container before initialization completes, requests will fail with 500 errors. Implementing a dedicated readiness probe ensures load balancers only direct traffic to containers that have fully initialized their model artifacts.

5.3 Maintenance, Data Drift, and Unexpected Behaviors

Once an AI microservice is deployed, its predictive quality degrades over time—a phenomenon known as model decay. Unlike traditional software bugs caused by bad code updates, model decay occurs while code remains entirely static.



1. Types of Drift

- ★ **Data Drift (Covariate Shift):** Occurs when the statistical distribution of input features changes over time while the relationship between features and target variables remains constant (e.g., macro-economic shifts altering income distributions in credit scoring models).
- ★ **Concept Drift:** Occurs when the underlying mathematical relationship between input features and target outcomes shifts completely (e.g., sudden changes in consumer purchasing behavior during unexpected global events).

2. Monitoring & Mitigation Strategies

To maintain deployed AI services over long lifecycles, operational teams must implement three essential safeguards:

1. **Asynchronous Feature Logging & Monitoring:** All incoming inference payloads should be logged asynchronously to analytical data warehouses (such as BigQuery or Snowflake). Statistical algorithms (like the Kolmogorov-Smirnov test or Population Stability Index) can then continuously compare live production feature distributions against historical training baselines.
2. **Shadow Deployments & Canary Releases:** When deploying updated model weights, teams should route production traffic to the new model in "shadow

mode"—executing predictions in parallel without returning them to the client—to compare latency and output stability against the live production baseline.

3. **Graceful Fallbacks & Rule-Based Circuit Breakers:** If an incoming request contains features outside expected numerical ranges, or if the model outputs a confidence score below a safety threshold (e.g., `< 0.50`), the API should gracefully fall back to deterministic heuristic rules rather than returning ungrounded predictions.

6. Project Verification & Conclusion

6.1 Verification Summary

The PoC microservice was containerized, executed, and validated inside Docker Desktop. HTTP requests dispatched to `/health` and `/predict` confirmed strict Pydantic parsing, correct numerical calculation, low processing latency (<10ms), and real-time log generation.